Programming for Robotics: Project Report

ATLAS: Autonomous Tracking Laser Aiming System

Author 1:	Madeline Abio s5259572
Author 2:	Alpa Sahai s5405889
GitHub Link:	https://github.com/alpasahai/programming_robotics_project.git
Specialisation Option Chosen:	Embedded Intelligence

Griffith University

Contents

List of Figures

State	Purpose
IDLE	• FCR is waiting for a cue from Search Radar • The turret idles with its search beam pointing in some specified direction. • **Exit conditions**: ▸ A cue is received from Search Radar (go to AIM) ▸ A ground hit cue is received by atlas_controller (go to RESET)
AIM	• After receiving a cue from Search Radar, but before finding the target itself, the FCR turret uses the most recent Search Radar updates to determine where to slew to. ▸ This may involve some kind of predictive or filtering feature if it is found experimentally that the Search Radar sensor noise is too unreliable or the projectile moves too quickly for the FCR to lock onto. Only add this complexity if it is deemed necessary. • This state is where the FCR is attempting to lock its tracking beam onto the target. The turret is moving. • No sensor fusion happens here because the only valid data points are the ones coming from the Search Radar. • **Exit conditions**: ▸ The FCR beam finds the target (go to TRACK & PREDICT) ▸ A ground hit cue is received by atlas_controller (go to RESET)
TRACK & PREDICT	• FCR is actively performing prediction using the Kalman filter which implements sensor fusion of the Search Radar and the FCR data points. This filter continually updates to make predictions on where the projectile will be in some timestep. • **Exit conditions**: ▸ The error between the predicted value and the observed value closes enough to justify making a prediction and firing a bullet (go to ENGAGING) ▸ A ground hit cue is received by atlas_controller (go to RESET)
ENGAGING	• The turret fires a bullet at the projectile based off the prediction it made. • No more predictions are happening here, the tracking can end. • **Exit conditions**: ▸ A projectile destroyed cue is received by atlas_controller (go to RESET) ▸ A ground hit cue is received by atlas_controller (go to RESET)
RESET	• Currently I'm not sure if there's really any reset logic, but maybe the Kalman filter memory should be wiped here (we may end up with multiple Kalman filters) • **Exit conditions**: ▸ System reset complete (go to IDLE)

Table 1: ATLAS Finite state machine states

1. Project Overview

Autonomous Tracking and Laser Aiming System (ATLAS) is a complete fire-control system capable of fully autonomous target identification, trajectory prediction and engagement. ATLAS consists of two units: the fire-control radar (FCR) and the search radar. These units operate as collaborative subsystems that communicate cueing information over emitters and receivers. The system was designed to operate in a simulated outdoor environment within Webots.

The role of the Search Radar is to continuously scan a wide search area and provide 3D radar cues to the FCR. The FCR consists of a turret-mounted narrow-beam radar and a projectile launch system used to engage incoming threats. Rotational motor actuators are used to control both the Search Radar sweep motion and the pan/tilt movement of the FCR turret, enabling the system to autonomously scan, track, and engage airborne targets. When a target is identified by the Search Radar, the FCR attempts to lock onto the target. Once the target has been locked, sensor data from both radar systems is fused using a Kalman filter to reduce measurement noise and estimate the projectile's future location. Once the error between the estimation and true location has been minimised, the projectile launch system attempts to neutralise the threat.

ATLAS uses online supervised learning—an SGDClassifier updated one launch at a time via `partial_fit`—to learn the attacker's launch-direction pattern live and pre-slew the turret toward the predicted next sector while idle, re-adapting continuously as the pattern drifts.

2. Technical Requirements Compliance

ATLAS is a complex system with more components and interfaces than are enumerated in this section. To keep the report concise, only the principal features that demonstrate alignment with each requirement are presented here.

2.1. General Requirements

2.1.1. Inputs

- World-frame target cue (x, y, z) from the external Search Radar process, delivered over a Webots radio link — the wide-area perception source that drives target acquisition.
- Turret-relative projectile position from ATLAS's own on-board Fire Control Radar, gated by range and FOV and produced only when locked — the precise perception source that drives tracking and engagement.

Additional inputs include pan and tilt joint-angle sensors that report the turret's true boresight, ground-hit and bullet-hit resolution pulses from the Attacker process, and the Webots simulation clock that drives the per-tick control loop.

2.1.2. Outputs

- Pan and tilt motor position commands to the turret's azimuth and elevation actuators, slewing the boresight onto the target.
- Bullet teleport-to-muzzle and velocity write toward the predicted intercept — the physical fire action that engages the threat.

2.1.3. FSM and Behavioural States

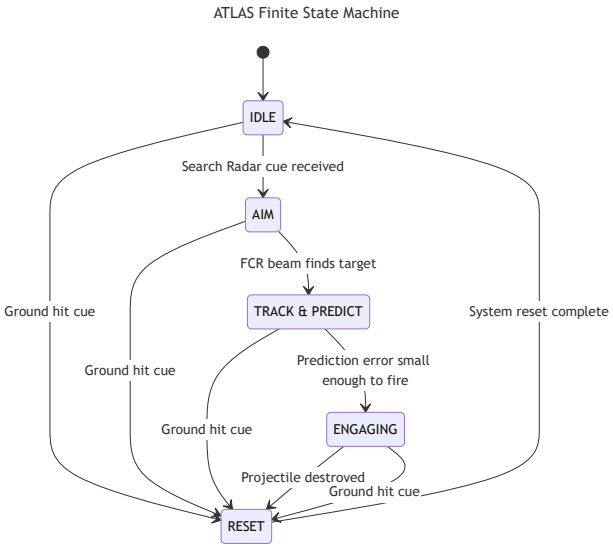

Figure 1: ATLAS finite state machine (FSM) diagram

Figure 1 shows ATLAS' FSM contains 5 behavioural states, demonstrating alignment with the FSM and minimum behavioural states requirement.

2.1.4. Multi-Condition Decision Logic

The transition labels in Figure 1 are summaries; the actual guards are multi-condition.

`TRACK_PREDICT` ⟶ `ENGAGING`: prediction error below threshold AND intercept in range AND sustained for N consecutive steps.

`IDLE` ⟶ `AIM`: cue present for N consecutive steps (debounced; single-frame flickers are rejected).

`ENGAGING` ⟶ `RESET`: ground-hit cue OR bullet-hit cue OR target out of range.

2.1.5. Safety or fail-safe mechanism

ATLAS implements two principal fail-safe mechanisms.

Convergence gate before firing: the turret will not fire on a single-frame "good prediction". `TRACK_PREDICT` only transitions to `ENGAGING` once the prediction error stays below threshold AND the intercept stays within range for `converge_frames` consecutive steps, preventing the system from committing a shot on a spurious low-error blip.

`RESET` recovery state: every state has a path to `RESET`, which wipes the Kalman filter, clears the stale Search Radar cue, and returns the FSM to `IDLE`. A corrupted track, a missed projectile, or any unexpected condition cannot persist across engagements. This ensures the system always recovers to a clean starting state before the next launch.

2.2. Embedded Intelligence Requirements

2.2.1. Sensor Fusion

A Kalman filter fuses two heterogeneous radar sources: the wide-area Search Radar (high noise, `R_SEARCH = 0.1`) and the narrow-beam FCR (low noise, `R_FCR = 0.001`). The filter weights precise FCR measurements far more heavily than coarse Search Radar cues. The predict step runs every tick regardless of measurement availability, so a sensor dropout does not collapse the estimate.

2.2.2. Context-aware Behaviour

The turret adapts to engagement context instead of following hard-coded triggers. In `IDLE`, the `AttackPredictor` (online logistic regression over discovered launch sectors) supplies a predicted next-sector bearing and the turret pre-slews toward it. The convergence gate also waits longer when the track is volatile and fires sooner when it is stable, rather than firing on a fixed timer.

2.2.3. Real-time Logic

ATLAS runs a synchronous Sense → Predict → Fuse → Decide loop on the Webots simulation clock (32 ms per tick). Sensor sampling, Kalman prediction, and FSM evaluation run every tick unconditionally, so the world model is always fresh when a decision is made.

3. System Architecture

ATLAs follows the Sense -> Process -> Decide -> Act architecture. As the information flows through the multiple modular subsystems that are responsible for perception, prediction, decision-making and physical actuation, the system acts accordingly. The architecture separates the behavioral logic, sensing logic, hardware control, and prediction into independent modules to improve robustness, system reliability, and maintainability.

The Sense layer focuses on the Search Radar and Fire Control Radar (FCR) subsystems. The Search Radar preforms wide-area scanning and broadcasts the coarse target cues to the FCR which performs the narrow-beam precision tracking once the projectile enters the turret's engagement region. The Process layer consists of the Kalman Filter and the Ballistic Trajectory Predictor. These help to estimate the projectile's position, velocity, and future interception points while reducing the sensor noise and improving stability.

The Decide layer is mainly the FSM logic. This is where the evaluation of the projectile trajectory confidence occurs along with the validity of prediction, engagement constraints, and the system's safety and exit conditions before determining the next behavioral state. Finally, the Act layer controls the physical behaviour of the turret (pan and tilt motor) and the bullet firing system. Once the engagement conditions have been met, the turret autonomously rotates and aligns with the predicted interception point and launches the bullet toward the target.

4. Behavioural Model

As mentioned, ATLAS uses a finite state machine (FSM) to manage autonomous target detection, tracking, prediction, and engagement. FSM control was selected because it provides deterministic behaviour, modular state transitions, and a reliable fail-safe recovery during the uncertain tracking conditions. ATLAS implements the following five behavioral states: IDLE, AIM, TRACK_PREDICT, ENGAGING, and RESET.

The system starts in the IDLE state where the Fire Control Radar (FCR) waits for the target cue information from the Search Radar subsystems. During this, the turret may pre-aim towards the attack sectors, predicted by the adaptive AttackPredictor module (get_ready_aim()). The Search Radar continuously broadcasts approximate projectile locations, and the FSM only transitions after a valid cue has been observed for multiple frames. The debounce logic prevents incorrect transitions caused by noisy or intermittent detections. Once stable cue is received from the Search Radar, the AIM state handles turret motion so that it slews towards the approximate projectile position. The FCR has not locked on to the target since the fused Kalman estimate is not yet reliable. The turret aims using raw cue measurements until the projectile enters the FCR's narrow field-of-view cone. When fcr.is_locked() becomes true, the TRACK_PREDICT stage begins.

The TRACK_PREDICT state preforms the main interception logic of the system. This state functions as the unified sensor-fusion and prediction pipeline as during this phase the Search Radar's measurements and the FCR's observations are combined using the Kalman-based TrackFilter. The Kalman filter continuously estimates projectile's position and velocity while reducing sensor noise and stabilising trajectory estimation. The turret tracks the filtered projectile position and computes the predictions for future intercept points; a prediction history is maintained throughout this process. This enables the system to compare previously predicted positions against the newly observed target positions. Once the prediction error remains consistent (below a predefined threshold), the prediction is considered reliable thus leading to the predicted intercept point being stored and FSM transitioning to ENGAGING.

During the ENGAGING state, the turret maintains aim on the fixed intercept point calculated during TRACK_PREDICT and fires a bullet toward the predicted intercept point. There are no predictions occurring during the engagement as the system assumes that the prediction has converged sufficiently. The ENGAGING stage terminates if projectile has been successfully intercepted and destroyed, projectile hits the ground, or the projectile has left the valid engagement range. The RESET stages act as a fail-safe recovery mechanism as it clears all the temporary engagement state that has accumulated during the previous interception cycle before returning to IDLE. This means resetting the Kalman Filter memory, clearing the stale Search Radar cues, resetting the prediction histories and clearing the intercept targets. This helps to prevent the stale sensor information from incorrectly triggering future engagements and allowing every interception cycle to begin from a clean system state.

The FSM architecture is essential in cleanly separating detection, aiming, tracking and prediction, engagement and recovery into independent behavioral stages. This approach improves the system's readability, maintainability, and robustness while ensuring reliable autonomous operation within a real-time robotics environment.

5. Perception / Sensor Processing

The system implements a dual-radar perception architecture made up of the Search Radar and a Fire-Control Radar (FCR). Since the native Webots Radar device only supports 2D radar measurements without elevation data, both radars were implemented as custom simulated 3D radar systems in Python. The radars are represented by a physical Webots node with a real position and orientation in the environment, while the detection logic itself is simulated through Supervisor access to projectile ground-truth positions.

The Search Radar is a wide-area acquisition radar that continuously scans the environment using a broader field-of-view (FOV). It periodically emits coarse world-frame projectile position cues with intentionally high noise added to the measurements. This creates long-range radar uncertainty while still allowing reliable target discovery. The Search Radar communicates these cues to the FCR through a cue-handoff process.

The FCR is implemented as a narrow-beam precision tracking radar mounted directly onto the turret. The FCR only monitors targets that fall within its restricted FOV cone and maximum engagement range. Once the turret slews toward the incoming Search Radar cue and the projectile enters the FCR cone, the system transitions into closed-loop tracking. The FCR produces lower-noise measurements, allowing for accurate predictions when following the projectile position.

Sensor measurements are processed using a Kalman-based TrackFilter which continuously predicts projectile motion and corrects the prediction using incoming FCR observations; this reduces sensor jitter and smooths noisy radar measurements. This allows the system to estimate projectile position

and velocity more reliably for future intercept prediction. The filtered projectile estimate is then used by the TRACK_PREDICT state to compute a ballistic intercept point ahead of the projectile trajectory. ATLAS also includes an adaptive attack pattern prediction feature. The attacker launches projectiles according to predefined frequency-based attack patterns with slight random variation. By observing projectile launch timing and trajectory behaviour over time, the system can pre-aim the turret toward regions where future launches are more likely to occur. This reduces acquisition delay and improves interception readiness during repeated attack sequences.

6. Safety and Robustness

ATLAS implements multiple safety and robustness mechanisms to ensure stable operation during uncertain conditions. The system uses confidence-gated engagement logic to prevent firing at the target when there is insufficient tracking or prediction confidence. It is ensured that engagement only occurs when the target has remained stable for multiple consecutive tracking updates.

We have implemented a protected firing sector restriction to prevent the turret from engaging with targets that fall within the restricted azimuth regions surrounding the Search Radar. While turret may continue to track target through full rotational movement, the firing commands are disabled within the restricted areas to prevent unsafe behaviour. By having sensor confidence, ATLAS showcases its safety and robustness by only engaging with targets when there is enough data to prove that the decision is confident. The target confidence must exceed a certain threshold, and prediction error must be low with the target remaining stable in order for firing to occur. The automatic RESET behaviour is triggered when the target is out of range; this means the sensor data has become stale, and invalid measurements can occur. The RESET state lets the systems return to a known safe state if abnormal behaviour were to occur. The Kalman filter supports the robustness of the system by reducing the noise so the sensor measurements and unstable target detections.

Additional mechanisms including the bullet-lifetime timeout, single-bullet interlocks, stable-cue clearing and target timeout handling further improve the safety of the system. These mechanisms unstable firing and overlapping engagements during unexpected simulation conditions.

7. Code Structure

The ATLAS software system is designed using modular architecture to separate all the independent components. The central layer that dictates the main process is implemented in atlas_controller.py. This is primarily responsible for constructing the system modules, wiring them together, and executing the main simulation loop.

Core perception and tracking functionalities are distributed across the FireControlRadar module (handling the narrow-field precision target tracking using field-of-view gating and range filtering) and the SearchRadarLink (receives target cues from external Search Radar controller). Projectile state estimation and prediction are handled by the TrackFilter (implements Kalman filtering) and the BallisticTrajectoryPredictor (computes future intercept locations for engagement). AtlasFSM controls the system's behaviour (managing IDLE, AIM, TRACK_PREDICT, ENGAGE, and RESET states). The adaptive behaviour is implemented through the AttackPredictor (analyses the attack-launch patterns and pre-slews the turret toward the future launch regions). Additional helper modules include: LaunchLatch, Bullet, Scoreboard, TrackTelemetry, AttackerGroundHItLink and BulletHitLink. These support projectile management, scoring, telemetry, and engagement landing.

The main control loop operates using Webot's fixed timestep execution model (robot.step(timestep) ! = −1) and during each timestep, the system follows Sense -> Process -> Decide -> Act architecture. Sense focuses on updating radar cues, projectile-hit signals, and FCR measurements. Process discusses the states of: Predict which focuses on the Kalman filter state estimate and Fuse which

focuses on integrating the new radar observations into the filter. Decide advances onto the FSM and determine the tracking and engagement actions. Act is moving the turret, firing the bullet toward the intercept point and updating the engagement state.

Several external Python libraries were used throughout this project. The Webots Python API (controller.Supervisor) provides access to robot devices, simulation state, and Supervisor functionality. numpy was used for numerical calculations and geometry handling, while filterpy provides the Kalman filtering framework used in projectile tracking. scikit-learn is used within the adaptive attack-pattern learner for lightweight machine-learning classification, and standard Python libraries such as math and os are used for geometry and configuration handling. The repository also includes pytest for automated unit testing and validation.

8. Key Code Snippits

9. Results / Demonstration Summary

ATLAS successfully demonstrates autonomous projectile detection, tracking, prediction, and interception within the Webots simulation environment. The system can detect a moving projectile and track them using the Search Radar process scans which send the coarse world-frame cues to the turret. The Fire Control Radar (FCR) located on the turret only locks onto the target when turret is physically skewed in that direction. The turret can use the Kalman track filter and ballistic trajectory predictor to estimate the projectile's future path and compute the intercept point. The result is a sensor-driven engagement pipleine where real projectile motion and collisions determine whether the turret bullet (fired when all the conditions are met) hits the incoming projectile, or the projectile hits the ground.

The system successfully demonstrates multiple safety and fail-safe behaviors including confidence-gated engagement, RESET recovery, and restricted firing sectors. These mechanisms implemented in ATLAS improve the reliability and robustness of the system, especially during unexpected circumstances like unstable sensor conditions and target loss. For this project, the main challenges faced include modeling the Search Radar and FCR as 3D radars in Webots environment, figuring out the incoming projectile's motion so the prediction and intercept pipeline functioned correctly, and adding the adaptive attack-pattern feature that learns launch sectors from observed cues.

10. Individual Contributions